ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INTRODUCTION TO ARTIFICIAL INTELLIGENCE(SENG4208)

INDIVISUAL ASSIGNMENT

NAME	ID NUMBER
Radyat Daniel	UGR/31115/15

Submitted to: Tagel Aboneh

Submission date:13/5/2025

1. **Define a game tree and explain its use in adversarial search problems.**

Game tree is a structured representation of all possible moves in a competitive (adversarial) game. It is a **directed graph**, where:

- **Nodes** represent game states.
- **Edges** represent player moves.
- **Root Node** is the initial state of the game.
- **Leaf Nodes** are terminal states (win, lose, or draw).
- **Levels** alternate between players (e.g., MAX vs. MIN in Minimax).

Adversarial Search and Its Use in Game Trees

Adversarial search is used between two plays competitive environments, where each player tries to maximize their own gain while minimizing the opponent's advantage. This applies to zero-sum games like chess, tic-tac-toe, or Go.

How Game Trees Help in Adversarial Search:

Predicting Opponent Moves:

- A game tree enumerates all possible actions for both players at each stage.
- Players can anticipate countermoves to make the best strategic decision.

Evaluating Decision Outcomes:

- Nodes are assigned scores based on how favorable they are to a player.
- Algorithms like Minimax traverse the tree to pick the optimal move.

Optimizing Strategy Using AI:

- Minimax Algorithm: Calculates the best possible move assuming the opponent plays optimally.
- Alpha-Beta Pruning: Improves efficiency by eliminating irrelevant branches in the tree.

2. **Differentiate between the Minimax algorithm and Alpha-Beta Pruning. In what scenarios does Alpha-Beta Pruning provide significant advantages?**

a. Minimax Algorithm

The Minimax algorithm is a decision-making strategy used in adversarial search, particularly in two-player games. It assumes that:

- The MAX player tries to maximize the game's score.
- The MIN player tries to minimize the game's score.
- The game tree is fully explored, and each move is evaluated based on potential wins/losses.
- The algorithm backtracks through the tree to decide the optimal move.

Example Use: Chess, Tic-Tac-Toe, Checkers, Go.

b. Alpha-Beta Pruning

Alpha-Beta Pruning is an optimized version of the Minimax algorithm. Instead of searching the entire game tree, it prunes (ignores) unnecessary branches that cannot affect the final decision.

Benefits:

- Reduces computation time by skipping irrelevant nodes.
- Works faster than standard Minimax.
- Allows deeper analysis with less processing power.

➢ Minimax explores all possible moves, while Alpha-Beta Pruning eliminates unneeded searches, making decision-making more efficient in strategic games.

When Alpha-Beta Pruning is Useful

1. Complex Games: Games with large search spaces, like chess and Go, benefit greatly from pruning.
2. Time Constraints: AI systems with limited computing power can evaluate deeper moves efficiently.
3. Competitive Strategy Games: Faster decision-making is critical for AI opponents in real-time games.

3. Explain game tree search algorithms and provide a clear example for each that illustrates how the algorithm works.

The most common algorithms are:

1. Minimax Algorithm
2. Alpha-Beta Pruning
3. Monte Carlo Tree Search (MCTS)

1. Minimax Algorithm

The Minimax algorithm is used in zero-sum games, where one player's gain is another player's loss. It aims to minimize the opponent's maximum possible gain while maximizing its own advantage.

Working Principle:

1. The game tree is fully expanded, showing all possible moves.
2. Players take turns choosing actions—MAX tries to maximize the score, while MIN tries to minimize it.
3. At terminal nodes, the algorithm assigns a score based on game outcomes (win = +1, loss = -1, draw = 0).
4. The tree is backtracked, selecting the best possible move for MAX while assuming MIN plays optimally.

Example: Tic-Tac-Toe (Partial Tree)

Consider a simplified Tic-Tac-Toe game where **MAX (X)** must choose between two moves:

X		
	O	

Possible Moves:

Move A (Top Center):

- If MAX plays here, MIN (O) responds optimally.

- Possible outcomes:
  - MIN blocks → Draw (0)
  - MIN makes a mistake → MAX wins (+1)
- Worst-case (optimal MIN play): 0 (Draw)

Move B (Bottom Left):

- If MAX plays here, MIN can force a win (-1).
- Worst-case: -1 (Loss)

Minimax Decision:

- MAX chooses Move A (0) because it guarantees at least a draw, unlike Move B (-1).

2. **Alpha-Beta Pruning**

Alpha-Beta pruning is an optimization of Minimax that reduces unnecessary computations by eliminating branches that cannot affect the final decision.

Working Principle:

1. Alpha (α) stores the best value for MAX so far.
2. Beta (β) stores the best value for MIN so far.
3. If a node produces a worse outcome than what has been seen before, the branch is pruned (ignored).
4. Pruning speeds up the decision-making process by skipping irrelevant calculations.

Example: Chess Game State

In a chess AI decision, if a possible move results in a situation where an already explored path guarantees a win, then the algorithm stops evaluating weaker moves.

If the first calculated path shows a guaranteed win, then further paths with lower scores are ignored, reducing computational time.

3. **Monte Carlo Tree Search (MCTS)**

Unlike Minimax and Alpha-Beta pruning, Monte Carlo Tree Search (MCTS) is a probabilistic approach that evaluates moves using random simulations instead of full tree expansion.

Working Principle:
Selection: Choose the most promising node based on a balance of exploration and exploitation.
Expansion: Add new possible moves to the tree.

Simulation: Run random games from the current position until reaching a terminal state.
Backpropagation: Update parent nodes with scores from simulations to refine future decisions.

Example: AI in Go

MCTS is used in Go-playing AI systems where exhaustive search is infeasible due to millions of possible moves. The AI simulates multiple games, learning which strategies have the highest probability of success.

Algorithm	Description	Best Use Case
Minimax	Full search of all possible moves, ensuring the best strategic play.	Chess, Tic-Tac-Toe, Checkers
Alpha-Beta Pruning	Optimized Minimax that eliminates unnecessary calculations.	Chess, Go (with deep search)
Monte Carlo Tree Search	Uses random simulations for decision-making.	Go, complex real-time strategy games

4. **Provide a step-by-step example showing how Minimax and Alpha-Beta Pruning evaluates nodes in a game tree and selects the optimal move.**

Step-by-Step Example: Minimax vs. Alpha-Beta Pruning in a Game Tree

Let's consider a simple two-player turn-based game where a player (MAX) must choose between different paths to maximize their final score, while the opponent (MIN) aims to minimize that score.

```
    A (MAX)
  /  |  \
```

B C D (MIN)

/ \ / \ / \

E F G H I J (MAX)

3 5 6 2 8 1

Minimax Evaluation (Full Search)

Step 1: Evaluate Terminal Nodes

- E=3, F=5, G=6, H=2, I=8, J=1 (given).

Step 2: Backpropagate Values (Bottom-Up)

- Level 2 (MIN Nodes B, C, D):
    - B: MIN chooses between E (3) and F (5) → 3.
    - C: MIN chooses between G (6) and H (2) → 2.
    - D: MIN chooses between I (8) and J (1) → 1.
- Level 1 (MAX Node A):
    - MAX chooses between B (3), C (2), D (1) → 3.

Final Decision

- Optimal Path: A → B → E (Score = 3).

3. Alpha-Beta Pruning (Optimized Search)

Uses α (best MAX) and β (best MIN) to skip unnecessary branches.

Step 1: Initialize α = -∞, β = +∞ at Root (A)

- A (MAX) explores children left to right.

Step 2: Evaluate Node B (MIN)

- B's children:
    - E=3 → MIN updates β = min (+∞, 3) = 3.
    - F=5 → 5 > 3 (β), so no update.
    - Pruning Condition: Since α(A) = -∞ < β(B) = 3, continue.
- B's final value = 3 → A updates α = max (-∞, 3) = 3.

Step 3: Evaluate Node C (MIN)

- C's children:
  - G=6 → 6 > β (current β = +∞), so β = 6.
  - H=2 → 2 < 6 → β = 2.
  - Pruning Check: Now, α(A) = 3 vs. β(C) = 2.
    Since α (3) > β (2), prune remaining siblings of C (no need to check D).

Step 4: Terminate Early

- D is never evaluated because C's value (2) is already worse than A's current α (3).

Final Decision

- Optimal Path: A → B → E (Score = 3), same as Minimax but with fewer evaluations.

Algorithm	Evaluated Nodes	Pruned Nodes	Total Checks
Minimax	E, F, G, H, I, J	None	6
Alpha-Beta Pruning	E, F, G, H	I, j	4

Both algorithms choose A→B→E (score=3), but Alpha-Beta does it faster. This example shows:

- **Minimax** is exhaustive but slow.
- **Alpha-Beta** is smarter, achieving the same result with less work.

Conclusion

Game trees play a fundamental role in artificial intelligence by providing structured decision-making frameworks for adversarial search problems. They enable AI systems to simulate strategic interactions, predict opponent behavior, and make optimal choices in competitive environments.
By applying search algorithms such as Minimax, Alpha-Beta Pruning, and Monte Carlo Tree Search (MCTS), AI can efficiently evaluate game states, improving performance in chess, Go, tic-tac-toe, and real-time strategy games. Alpha-Beta Pruning enhances computational efficiency by eliminating unnecessary searches, while Monte Carlo Tree Search handles uncertainty with probabilistic simulations. Beyond gaming, game tree methodologies are widely used in robotics, decision automation, and AI-driven strategic planning.